Update Data - UPDATE

Now let's see how to update data using **SQLModel**.

Continue From Previous Code

As before, we'll continue from where we left off with the previous code.

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Remember to remove the `database.db` file before running the examples to get the same results.

Update with SQL

Let's quickly check how to update data with SQL:

```
UPDATE hero
SET age=16
WHERE name = "Spider-Boy"
```

This means, more or less:

> Hey SQL database 👋, I want to `UPDATE` the table called `hero`.
>
> Please `SET` the value of the `age` column to `16` ...
>
> ...for each of the rows `WHERE` the value of the column `name` is equal to `"Spider-Boy"`.

In a similar way to `SELECT` statements, the first part defines the columns to work with: what are the columns that have to be updated and to which value. The rest of the columns stay as they were.

And the second part, with the `WHERE`, defines to which rows it should apply that update.

In this case, as we only have one hero with the name `"Spider-Boy"`, it will only apply the update in that row.

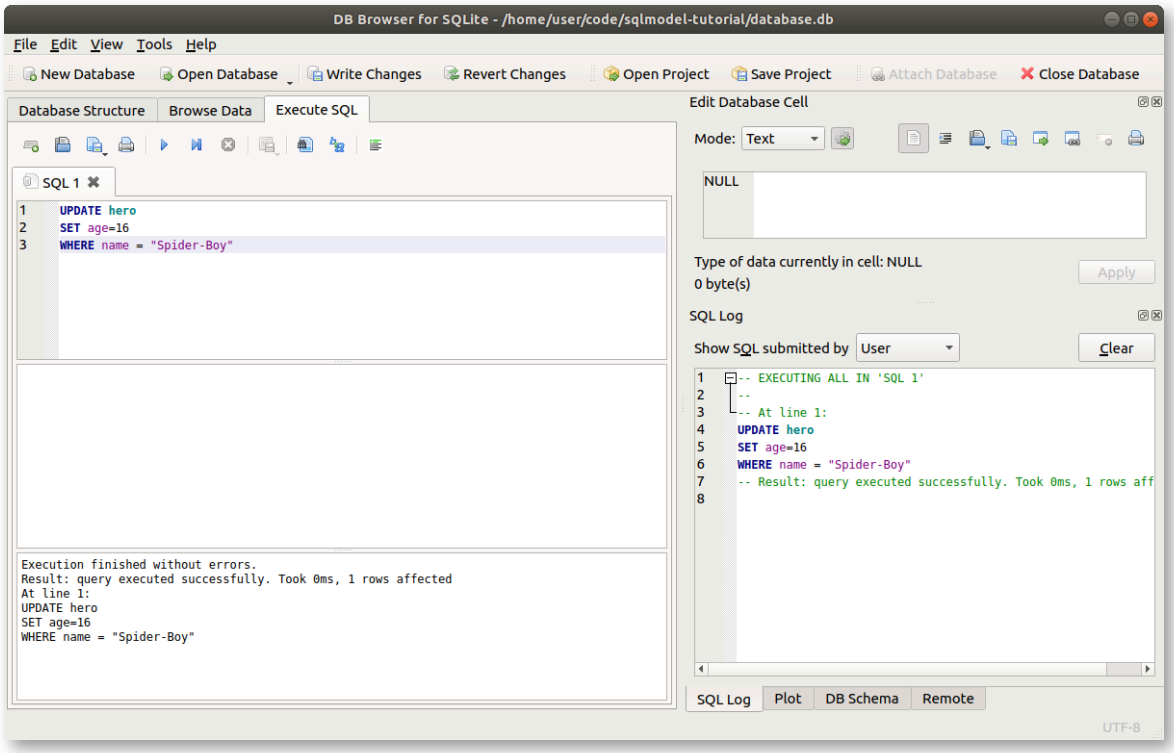
Info

Notice that in the `UPDATE` the single equals sign (`=`) means **assignment**, setting a column to some value.

And in the `WHERE` the same single equals sign (`=`) is used for **comparison** between two values, to find rows that match.

This is in contrast to Python and most programming languages, where a single equals sign (`=`) is used for assignment, and two equal signs (`==`) are used for comparisons.

You can try that in **DB Browser for SQLite**:



After that update, the data in the table will look like this, with the new age for Spider-Boy:

id	name	secret_name	age

1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	16 ✨
3	Rusty-Man	Tommy Sharp	48

Tip

It will probably be more common to find the row to update by `id`, for example:

```
UPDATE hero
SET age=16
WHERE id = 2
```

But in the example above I used `name` to make it more intuitive.

Now let's do the same update in code, with **SQLModel**.

To get the same results, delete the `database.db` file before running the examples.

Read From the Database

We'll start by selecting the hero `"Spider-Boy"`, this is the one we will update:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Let's not forget to add that `update_heroes()` function to the `main()` function so that we call it when executing the program from the command line:

Python 3.10+

```python
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Up to that point, running that in the command line will output:

```bash
```

Tip

Notice that by this point, the hero still doesn't have an age.

Set a Field Value

Now that you have a `hero` object, you can simply set the value of the field (the attribute representing a column) that you want.

In this case, we will set the `age` to `16`:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Add the Hero to the Session

Now that the hero object in memory has a change, in this case a new value for the `age`, we need to add it to the session.

This is the same we did when creating new hero instances:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Commit the Session

To save the current changes in the session, **commit** it.

This will save the updated hero in the database:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

It will also save anything else that was added to the session.

For example, if you were also creating new heroes and had added those objects to the session before, they would now be saved too in this single commit.

This commit will generate this output:

```bash
```

Refresh the Object

At this point, the hero is updated in the database and it has the new data saved there.

The data in the object would be automatically refreshed if we accessed an attribute, like `hero.name`.

But in this example we are not accessing any attribute, we will only print the object. And we also want to be explicit, so we will `.refresh()` the object directly:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

This refresh will trigger the same SQL query that would be automatically triggered by accessing an attribute. So it will generate this output:

```bash
```

Print the Updated Object

Now we can just print the hero:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

        hero.age = 16
        session.add(hero)
        session.commit()
        session.refresh(hero)
        print("Updated hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Because we refreshed it right after updating it, it has fresh data, including the new `age` we just updated.

So, printing it will show the new `age`:

```bash
```

Review the Code

Now let's review all that code:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()
```

```python
def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")  1
        results = session.exec(statement)  2
        hero = results.one()  3
        print("Hero:", hero)  4

        hero.age = 16  5
        session.add(hero)  6
        session.commit()  7
        session.refresh(hero)  8
        print("Updated hero:", hero)  9


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

1. Select the hero we will work with.
2. Execute the query with the select statement object.

   This generates the output:

   ```
   INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
   FROM hero
   WHERE hero.name = ?
   INFO Engine [no key 0.00017s] ('Spider-Boy',)
   ```

3. Get one hero object, expecting exactly one.

   > **Tip**
   >
   > This ensures there's no more than one, and that there's exactly one, not `None`.
   >
   > This would never return `None`, instead it would raise an exception.

4. Print the hero object.

   This generates the output:

   ```
   Hero: name='Spider-Boy' secret_name='Pedro Parqueador' age=None id=2
   ```

5. Set the hero's age field to the new value `16`.

   Now the `hero` object in memory has a different value for the age, but it is still not saved to the database.

6. Add the hero to the session.

   This puts it in that temporary place in the session before committing.

   But it's still not saved in the database yet.

7. Commit the session.

   This saves the updated hero to the database.

   And this generates the output:

   ```
   INFO Engine UPDATE hero SET age=? WHERE hero.id = ?
   INFO Engine [generated in 0.00017s] (16, 2)
   INFO Engine COMMIT
   ```

8. Refresh the hero object to have the recent data, including the age we just committed.

   This generates the output:

```
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.id = ?
INFO Engine [generated in 0.00018s] (2,)
```

9. Print the updated hero object.

   This generates the output:

```
Updated hero: name='Spider-Boy' secret_name='Pedro Parqueador' age=16 id=2
```

Tip

Check out the number bubbles to see what is done by each line of code.

Multiple Updates

The update process with **SQLModel** is more or less the same as with creating new objects, you add them to the session, and then commit them.

This also means that you can update several fields (attributes, columns) at once, and you can also update several objects (heroes) at once:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")  1
        results = session.exec(statement)  2
        hero_1 = results.one()  3
        print("Hero 1:", hero_1)  4

        statement = select(Hero).where(Hero.name == "Captain North America")  5
        results = session.exec(statement)  6
        hero_2 = results.one()  7
        print("Hero 2:", hero_2)  8

        hero_1.age = 16  9
        hero_1.name = "Spider-Youngster"  10
        session.add(hero_1)  11

        hero_2.name = "Captain North America Except Canada"  12
        hero_2.age = 110  13
        session.add(hero_2)  14

        session.commit()  15
        session.refresh(hero_1)  16
        session.refresh(hero_2)  17

        print("Updated hero 1:", hero_1)  18
        print("Updated hero 2:", hero_2)  19
    20

# Code below omitted 👇
```

1. Select the hero `Spider-Boy`.
2. Execute the select statement.

   This generates the output:

```
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.name = ?
INFO Engine [no key 0.00018s] ('Spider-Boy',)
```

3. Get one hero object, the only one that should be there for **Spider-Boy**.
4. Print this hero.

   This generates the output:

```
Hero 1: name='Spider-Boy' secret_name='Pedro Parqueador' age=None id=2
```

5. Select another hero.
6. Execute the select statement.

   This generates the output:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.name = ?
INFO Engine [no key 0.00020s] ('Captain North America',)
```

> **Tip**
>
> See the `BEGIN` at the top?
>
> This is SQLAlchemy automatically starting a transaction for us.
>
> This way, we could revert the last changes (if there were some) if we wanted to, even if the SQL to create them was already sent to the database.

7. Get one hero object for this new query.

   The only one that should be there for **Captain North America**.
8. Print this second hero.

   This generates the output:

```
Hero 2: name='Captain North America' secret_name='Esteban Rogelios' age=93 id=7
```

9. Update the age for the first hero.

   Set the value of the attribute `age` to `16`.

   This updates the hero object in memory, but not yet in the database.
10. Update the name of the first hero.

    Now the name of the hero will not be `"Spider-Boy"` but `"Spider-Youngster"`.

    Also, this updates the object in memory, but not yet in the database.
11. Add this first hero to the session.

    This puts it in the temporary space in the **session** before committing it to the database.

    It is not saved yet.
12. Update the name of the second hero.

Now the hero has a bit more precision in the name. 😜

This updates the object in memory, but not yet in the database.

13 Update the age of the second hero.

This updates the object in memory, but not yet in the database.

14 Add the second hero to the session.

This puts it in the temporary space in the **session** before committing it to the database.

15 Commit all the changes tracked in the session.

This commits everything in one single batch.

This generates the output:

```
INFO Engine UPDATE hero SET name=?, age=? WHERE hero.id = ?
INFO Engine [generated in 0.00028s] (('Spider-Youngster', 16, 2), ('Captain North America
Except Canada', 110, 7))
INFO Engine COMMIT
```

Tip

See how SQLAlchemy (that powers SQLModel) optimizes the SQL to do as much work as possible in a single batch.

Here it updates both heroes in a single SQL query.

16 Refresh the first hero.

This generates the output:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.id = ?
INFO Engine [generated in 0.00023s] (2,)
```

Tip

Because we just committed a SQL transaction with `COMMIT`, SQLAlchemy will automatically start a new transaction with `BEGIN`.

17 Refresh the second hero.

This generates the output:

```
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
WHERE hero.id = ?
INFO Engine [cached since 0.001709s ago] (7,)
```

Tip

SQLAlchemy is still using the previous transaction, so it doesn't have to create a new one.

18 Print the first hero, now updated.

This generates the output:

```
Updated hero 1: name='Spider-Youngster' secret_name='Pedro Parqueador' age=16 id=2
```

19 Print the second hero, now updated.

This generates the output:

```
Updated hero 2: name='Captain North America Except Canada' secret_name='Esteban Rogelios'
age=110 id=7
```

20 Here is the end of the `with` block statement, so the session can execute its terminating code.

The session will `ROLLBACK` (undo) any possible changes in the last transaction that were not committed.

This generates the output:

```
INFO Engine ROLLBACK
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")  # (1)!
        results = session.exec(statement)  # (2)!
        hero_1 = results.one()  # (3)!
        print("Hero 1:", hero_1)  # (4)!

        statement = select(Hero).where(Hero.name == "Captain North America")  # (5)!
        results = session.exec(statement)  # (6)!
        hero_2 = results.one()  # (7)!
        print("Hero 2:", hero_2)  # (8)!

        hero_1.age = 16  # (9)!
        hero_1.name = "Spider-Youngster"  # (10)!
        session.add(hero_1)  # (11)!

        hero_2.name = "Captain North America Except Canada"  # (12)!
        hero_2.age = 110  # (13)!
        session.add(hero_2)  # (14)!

        session.commit()  # (15)!
        session.refresh(hero_1)  # (16)!
        session.refresh(hero_2)  # (17)!

        print("Updated hero 1:", hero_1)  # (18)!
        print("Updated hero 2:", hero_2)  # (19)!
    # (20)!
```

```python
def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Tip

Review what each line does by clicking each number bubble in the code. 👆

Recap

Update **SQLModel** objects just as you would with other Python objects. 🐍

Just remember to `add` them to a **session**, and then `commit` it. And if necessary, `refresh` them.